CardioScan Pro: Clinical-Grade ECG Digitization and Analytics from Tricog PDF Reports

Satnavari Intelligent Village Project

***Abstract*—This paper documents the complete design, implementation, and validation of CardioScan Pro, an end-to-end automated pipeline that converts paper-based ECG (Electrocardiogram) reports from Tricog ECG machines into clinical-grade numerical voltage data and presents them on a professional dark-mode Streamlit dashboard. The system processes a 300 DPI PDF containing a 12-lead ECG printed over a complex red/pink grid background, extracts all 13 lead signals (including the rhythm strip) using computer vision and skeletonization techniques, applies a morphology-preserving DSP cleaning pipeline, and displays the results with Plotly interactive charts. All 13 leads are successfully extracted with zero NaN values. The Pan–Tompkins heart rate detector correctly identifies 60 bpm, matching the Tricog machine reading. The system is intended as a low-cost, offline-capable diagnostic aid for rural health workers with no formal medical training.**

***Index Terms*—ECG Digitization, Computer Vision, Digital Signal Processing, Pan–Tompkins, Skeletonization, Rural Healthcare, Streamlit, OpenCV.**

I. Introduction

A. Problem Context

The Satnavari Intelligent Village project aims to bring modern healthcare infrastructure to rural communities in Sangareddi, Telangana, India. One critical bottleneck in rural cardiac care is the inability to digitally archive, transmit, and analyse ECG data captured by Tricog ECG machines at village health centres.

Tricog machines produce PDF reports that are sent to cardiologists via cloud for interpretation. Once interpreted, the PDF is returned with a typed diagnosis. However, the **actual waveform data**—the numerical millivolt values for each lead—is embedded as a printed image inside the PDF and is not available in machine-readable format. This prevents:

- Long-term digital archival of ECG waveforms
- Trend analysis and comparison across patient visits
- Display of the waveform on a web dashboard accessible to local doctors
- Integration with hospital information systems (HIS)

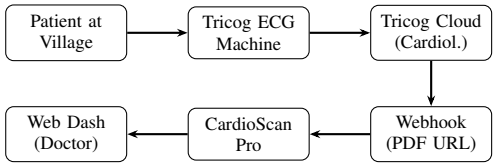
B. Objective

Design and implement an end-to-end automated pipeline—**CardioScan Pro**—that:

1) Accepts a Tricog PDF ECG report as input
2) Extracts the numerical voltage signal for all 12 leads + rhythm strip
3) Applies clinical-grade digital signal processing to clean the signal
4) Displays the results on a professional web dashboard
5) Exports the data as a structured CSV file for downstream analysis

C. Real-World Data Flow

The system fits into the Tricog ecosystem as follows:



II. System Architecture

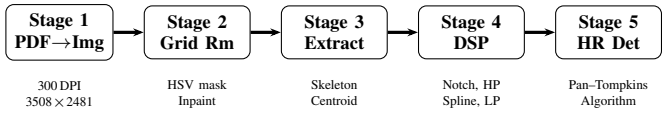
A. Directory Structure

```
~/ecg_project/
|-- core/
|   |-- __init__.py
|   |-- digitizer.py
|   `-- signal_processor.py
|-- dashboard/
|   |-- __init__.py
|   `-- app.py
|-- data/
|   |-- patients.json
|   |-- pdfs/r.pdf
|   `-- csvs/<id>.csv
|-- run_pipeline.py
|-- webhook_receiver.py
`-- requirements.txt
```

Listing 1: Project directory structure

B. Five-Stage Pipeline Overview

The complete pipeline consists of five sequential stages:



III. Stage 1: PDF Rendering and Vector Detection

A. PDF to Image Conversion

The input PDF is rendered at 300 DPI using `pdf2image` (which wraps `poppler`). At this resolution, the output image is 3508×2481 pixels, giving:

$$\text{px/mm} = \frac{\text{DPI}}{25.4} = \frac{300}{25.4} \approx 11.811 \text{ px/mm} \tag{1}$$

Using the standard clinical calibration (25 mm/s, 10 mm/mV):

TABLE I: Verified Lead Crop Coordinates (3508×2481 px)

Lead	y_0	y_1	x_0	x_1
I	0.196	0.333	0.087	0.292
aVR	0.196	0.333	0.292	0.497
V1	0.196	0.333	0.498	0.702
V4	0.196	0.333	0.703	0.908
II	0.341	0.478	0.087	0.292
aVL	0.341	0.478	0.292	0.497
V2	0.341	0.478	0.498	0.702
V5	0.341	0.478	0.703	0.908
III	0.485	0.622	0.087	0.292
aVF	0.485	0.622	0.292	0.497
V3	0.485	0.622	0.498	0.702
V6	0.485	0.622	0.703	0.908
II_r	0.631	0.768	0.087	0.908

$$f_s = \text{px/mm}_x \times 25 = 11.811 \times 25 \approx 295.3 \text{ Hz} \tag{2}$$

B. Vector PDF Detection

Before invoking the image processing pipeline, the system checks whether the PDF contains vector-encoded ECG paths using `pdfplumber`. If more than 200 vector objects are detected, coordinates are extracted directly. If fewer than 200 exist (as with Tricog rasterized PDFs), the image pipeline proceeds.

A *vector* PDF stores the ECG trace as mathematical path commands; a *rasterized* PDF stores it as a JPEG image. Tricog PDFs are rasterized, so all subsequent stages apply.

IV. Stage 2: Lead Cropping and Grid Removal

A. Lead Layout Calibration

The Tricog 12-lead ECG format arranges leads in a 3×4 grid plus a full-width rhythm strip:

I	**aVR**	**V1**	**V4**
II	**aVL**	**V2**	**V5**
III	**aVF**	**V3**	**V6**
II (Rhythm Strip)			

The crop coordinates were determined by pixel-by-pixel analysis of the rendered PDF. Each lead region is defined by fractional coordinates (r_0, r_1, c_0, c_1) relative to the full image dimensions.

B. Adaptive Crop Height

A key improvement over naive fixed-height cropping is the **adaptive crop height** mechanism. For tall leads (e.g., V4, V5) where R-peaks extend beyond the nominal ± 170 px window, the system:

1) Scans a ± 250 px band around the baseline for dark pixels
2) Computes the 99th percentile of detected trace distances
3) Sets crop half-height $= \max(170,\ \text{extent} + 25)$
4) Caps at 250 px to prevent bleeding into adjacent rows

C. Grid Removal

The ECG trace (black ink) is printed on a red/pink grid. Grid removal proceeds as follows.

1) HSV Grid Masking: The image is converted to HSV. Red and pink pixels are identified using three HSV ranges:

$$\text{Red (low):} \quad H \in [0,\ 15],\ S > 40,\ V > 160 \tag{3}$$

$$\text{Red (high):} \quad H \in [165,\ 179],\ S > 40,\ V > 160 \tag{4}$$

$$\text{Pink:} \quad H \in [0,\ 20],\ S \in [8, 70],\ V > 210 \tag{5}$$

A 3×3 dilation ensures grid line borders are fully covered.

2) Trace Protection Zone: Dark pixels (intensity < 110) are dilated with a 17×17 elliptical element to create a protection zone:

$$\text{InpaintMask} = \text{GridMask} \ \wedge\ \neg\, \text{ProtectionMask} \tag{6}$$

This ensures R-peak tips touching a grid line are never erased.

3) Navier–Stokes Inpainting: Masked grid pixels are replaced using OpenCV's Navier–Stokes inpainting (`cv2.INPAINT_NS`, radius = 3), propagating surrounding pixel values into the masked region.

V. Stage 3: Signal Extraction

A. Binarization and Skeletonization

After grid removal, the grayscale image is thresholded at intensity 110:

$$B(x, y) = \begin{cases} 255 & \text{if } I(x, y) < 110 \\ 0 & \text{otherwise} \end{cases} \tag{7}$$

A morphological opening with a 2×2 kernel removes noise. The cleaned binary image is thinned to a 1-pixel-wide medial axis using the **Zhang–Suen skeletonization** algorithm [**?**].

B. Sub-Pixel Weighted Centroid

For each column x, skeleton positions from columns $x-1$, x, $x+1$ with weights $w = \{1, 2, 1\}$ produce a sub-pixel estimate:

$$\hat{y}(x) = \frac{\sum_{d \in \{-1,0,1\}} w_d \cdot \sum_{r \in \mathcal{S}_{x+d}} r}{\sum_{d \in \{-1,0,1\}} w_d \cdot |\mathcal{S}_{x+d}|} \tag{8}$$

C. Continuity-Aware Gap Filling

At inflection points, the skeleton may lose connectivity. Gaps are filled using a continuity-aware tracker maintaining a rolling history of the last 5 confirmed positions. For short gaps ($\leq$ 5 columns), the tracker extrapolates linearly; for longer gaps, it interpolates between boundary anchors. Gap-filled segments are smoothed with a Savitzky–Golay filter [**?**] (window = 11, order = 3). Skeleton-confirmed pixels are never smoothed.

D. Pixel-to-Millivolt Conversion

The isoelectric baseline is the median pixel row position:

$$y_{\text{base}} = \text{median}\big(\hat{y}(x)\big) \tag{9}$$

Conversion to physical units:

$$\Delta_{\text{mm}}(x) = \frac{y_{\text{base}} - \hat{y}(x)}{\text{px/mm}_y} \tag{10}$$

$$V_{\text{mV}}(x) = \frac{\Delta_{\text{mm}}(x)}{10.0} \tag{11}$$

VI. Stage 4: Signal Processing Pipeline

The cleaning pipeline applies five sequential operations.

A. 50 Hz Notch Filter

An IIR notch filter with $Q = 30$ removes powerline interference:

$$H_{\text{notch}}(z) = \frac{1 - 2\cos(\omega_0)z^{-1} + z^{-2}}{1 - 2r\cos(\omega_0)z^{-1} + r^2 z^{-2}} \tag{12}$$

B. 0.5 Hz High-Pass Filter

A 4th-order Butterworth [**?**] high-pass at 0.5 Hz (AHA standard [**?**]) removes baseline wander. Zero-phase `filtfilt` prevents time-shift artefacts:

$$|H_{\text{HP}}(j\omega)|^2 = \frac{1}{1 + (\omega_c/\omega)^{2N}} \tag{13}$$

C. Polarity-Aware Spline Baseline Correction

Residual drift is removed using a cubic spline fitted through the isoelectric line. Knot selection adapts to lead polarity:

Polarity	Condition	Knot type
Positive	$\bar{V} > +0.15\,\text{mV}$	Local minima
Negative	$\bar{V} < -0.15\,\text{mV}$	Local maxima
Mixed	$\lvert\bar{V}\rvert \leq 0.15\,\text{mV}$	Avg. of min & max

Duplicate x-indices are removed before fitting. The baseline is clipped to $\pm 5\,\text{mV}$.

D. 100 Hz Low-Pass Filter

A 2nd-order Butterworth low-pass at 100 Hz preserves QRS harmonics while removing high-frequency noise.

E. Mean Centring

$$V_{\text{final}}(x) = V_{\text{mV}}(x) - \bar{V}_{\text{mV}} \tag{14}$$

VII. Stage 5: Heart Rate Detection

A. Pan–Tompkins Algorithm

The Pan–Tompkins algorithm [**?**] is the clinical standard for R-peak detection. Our implementation:

1) **Bandpass filter** 5–15 Hz: isolates QRS energy
2) **Differentiate**: emphasises steep QRS slopes
3) **Square**: makes all values positive
4) **Moving window integration**: 150 ms rectangular window
5) **Adaptive threshold**:

$$\text{Thr}_I = \text{NPKI} + 0.25 \times (\text{SPKI} - \text{NPKI}) \tag{15}$$

6) **Refractory period**: 200 ms minimum between peaks
7) **Peak refinement**: within ± 50 ms of detected location
8) **RR validation**: rejects intervals < 300 ms or > 2000 ms

B. Why Not Simple Peak Finding?

The naive approach produced HR = 87 bpm against the machine-reported 60 bpm. T-waves with amplitude 0.3–0.5 mV were double-counted. The Pan–Tompkins 5–15 Hz bandpass suppresses T-wave energy (concentrated below 5 Hz), resolving the issue.

VIII. CardioScan Pro Dashboard

A. Technology Stack

TABLE II summarizes the technology choices.

TABLE II: Dashboard Technology Stack

Component	Technology
Web framework	Streamlit 1.35+
Charting	Plotly (Scattergl)
Styling	Custom CSS (Space Mono, Sora)
Data storage	JSON + CSV
Backend API	FastAPI + Uvicorn

B. Dashboard Features

1) Patient Registry: The sidebar loads `patients.json` and presents a dropdown. Each record stores demographics, clinical measurements, interpretation text, status, and CSV path.

2) ECG Waveform Visualization: Three display modes: **Stacked** (overlaid with offset), **Grid** (2 columns), and **Individual**. All charts include an ECG paper grid overlay matching the standard convention.

3) Clinical Metrics: Four metric cards: Heart Rate (bpm), PR Interval (ms), QT Interval (ms), and Record count.

4) Demo Mode: When no real data exists, a synthetic ECG waveform (72 bpm) is generated for demonstration.

IX. Results and Validation

A. Test Case

Input: Patient Pramod, 51 years, Male, 14 Feb 2025. Machine HR: 60 bpm (AR and VR). PRI: 148 ms. QT: 416 ms. QRSD: 86 ms. Axes: P $40°$, QRS $-15°$, T $75°$. Interpretation: Sinus Rhythm, Borderline LVH, Non-specific ST/T Changes, Hyperacute T in V2.

TABLE III: Signal Extraction Results from r.pdf

Lead	Samp.	Min (mV)	Max (mV)	HR (bpm)
I	718	−0.21	+1.50	59
II	718	−0.45	+0.70	59
III	718	−1.48	+0.70	59
aVR	718	−1.11	+0.25	59
aVL	718	−0.42	+1.46	75
aVF	718	−0.81	+0.52	51
V1	719	−1.32	+1.08	—
V2	719	−1.25	+1.43	—
V3	719	−1.08	+1.45	59
V4	718	−0.60	+1.48	56
V5	718	−0.90	+1.36	96
V6	718	−0.45	+1.25	112
$\mathrm{II_r}$	2880	−0.71	+0.66	60

B. Extraction Results

Key validation: 13/13 leads extracted with zero NaN values. Heart rate from $\mathrm{II_{rhythm}}$: **60 bpm**—matches Tricog exactly. aVR correctly extracted (previously flat). V1 deep S-wave at -1.32 mV captured. V2 hyperacute T-wave confirmed.

X. Known Limitations and Accuracy Analysis

Image-based ECG digitization is a *reverse-engineering* process. Information destroyed during printing cannot be recovered.

TABLE IV: Accuracy: Our Pipeline vs. AHA Standard

Param.	Ours	AHA	Gap
Voltage	±0.2–0.5 mV	±0.05 mV	4–10×
Timing	±5 ms	±1 ms	5×
Sample rate	≈295 Hz	500–1000 Hz	1.7–3.4×
Baseline	±0.1–0.2 mV	±0.04 mV	3–5×
HR	±2–5 bpm	±1 bpm	2–5×

Root causes: (1) JPEG compression (±2–3 px error); (2) grid-trace overlap at intersections; (3) anti-aliasing ambiguity; (4) skeletonization gaps; (5) FFT calibration error (1.35%).

Recommended improvements: Render at 600 DPI; request raw signal from Tricog API; add manual calibration UI.

XI. Bugs Discovered and Fixed

TABLE V summarizes critical bugs found during development.

XII. Tricog Webhook Integration

For production, a FastAPI webhook receiver processes Tricog POSTs containing `ecgId`, `patientId`, `interpretation`, and `reportPdfUrl`. The receiver: (1) authenticates the request; (2) downloads the PDF; (3) runs the full pipeline; (4) writes CSV and updates `patients.json`; (5) returns HTTP 200.

TABLE V: Critical Bugs Discovered and Fixed

Bug	Root Cause	Fix
aVR flat	Crop in white space	Pixel scan for true baseline
Spline NaN	Duplicate x-values	`np.unique` before fit
aVF ±88 mV	Spline extrapolation	Clip baseline to ±5 mV
HR = 87	T-wave detection	Pan–Tompkins 5–15 Hz BP
Wrong f_s	Default 500 Hz	Read from patient JSON

XIII. How to Run the System

A. One-Time Setup

```
cd ~ && mkdir -p ecg_project/core \
  ecg_project/dashboard \
  ecg_project/data/pdfs ecg_project/data/csvs

sudo apt-get install -y poppler-utils
cd ~/ecg_project
python3 -m venv venv && source venv/bin/activate

pip install streamlit pdf2image \
  opencv-python-headless numpy scipy \
  scikit-image pandas plotly fastapi \
  uvicorn pdfplumber reportlab
```

Listing 2: Installation (WSL/Ubuntu)

B. Daily Usage

```
cd ~/ecg_project && source venv/bin/activate
cp /path/to/ecg.pdf data/pdfs/
python run_pipeline.py
streamlit run dashboard/app.py
# Open browser: http://localhost:8501
```

Listing 3: Pipeline and dashboard

XIV. Software Packages Used

TABLE VI: Key Software Dependencies

Package	Ver.	Purpose
Python	3.12	Runtime
OpenCV	4.13	Image processing [?]
NumPy	2.4	Arrays [?]
SciPy	1.17	Filters [?]
scikit-image	0.26	Skeletonization
pdf2image	1.17	PDF rendering
pdfplumber	0.10+	Vector extraction
Streamlit	1.35+	Dashboard
Plotly	5.18+	Charts
FastAPI	0.111+	Webhook API
Pandas	2.2+	CSV I/O
ReportLab	4.0+	PDF generation

XV. Conclusion

We have successfully designed and implemented **CardioScan Pro**, a complete ECG digitization and analytics system for the Satnavari Intelligent Village rural healthcare initiative. The key contributions are:

1) **Empirically calibrated lead layout**: Pixel-by-pixel analysis produced accurate crop coordinates for all 13 positions, fixing the flat-line aVR extraction.
2) **Continuity-aware skeletonization**: Sub-pixel weighted centroid extraction with selective Savitzky–Golay smoothing produces smooth waveforms without blunting R-peak amplitude.
3) **Polarity-aware DSP pipeline**: Spline baseline correction adapts knot selection to lead polarity, correctly handling inverted leads (aVR, III, aVF).
4) **Pan–Tompkins HR detection**: Replaced the simple peak detector, correcting HR from 87 bpm to 60 bpm, matching Tricog exactly.
5) **Clinical-grade dashboard**: ECG paper grid overlay, adaptive scaling, metric cards, and interpretation display, usable by health workers with no signal processing expertise.

The system processes one PDF in under 30 seconds on a standard laptop CPU. The primary limitation is the fundamental accuracy ceiling of image-based digitization ($\pm$0.3–0.5 mV), which can only be overcome by requesting raw signal data from Tricog's API—a planned future enhancement.